

Výstupy ŠVP

INF-07a Ve vztahu k charakteru a velikosti vstupu hodnotím nároky algoritmů; porovnám algoritmy podle různých hledisek a vyberu pro řešený problém ten nejvhodnější. (*Informatika 4, 2/2 · z INF-07*)

INF-07b Vylepším algoritmus podle zvoleného hlediska a zobecním řešení pro širší třídu problémů. (*Informatika 4, 2/2 · z INF-07*)

INF-08d Ve snaze o vyšší efektivitu navrhuji, řídím a hodnotím souběh procesů. (*Informatika 4, 2/2 · z INF-08*)

Učivo: testování, optimalizace – vliv vstupních dat na spotřebované výpočetní zdroje

Připomenutí

$$2^k \geq n + 1$$

$$\frac{n(n-1)}{2}$$

$$n \log_2 n$$

$$S = \frac{t_1}{t_p}$$

$$t_p = t_{\text{sekv}} + \frac{t_{\text{dělit}}}{p}$$

Efektivita algoritmů: lineární a binární vyhledávání, dva algoritmy řazení, vylepšení a zobecnění algoritmu a souběh procesů. Počty kroků si ověřuj programem, časy měř pro několik velikostí vstupu. Podrobnosti najdeš v listu *Must know*.

A. Rozcvička

- Kolik porovnání nejvýše potřebuje lineární a kolik binární vyhledávání v seřazeném seznamu se 100, 10 000 a 1 000 000 prvky? ★☆☆
- Ručně proved' binární vyhledávání čísla 34 a čísla 5 v seznamu [3, 8, 12, 17, 21, 25, 30, 34, 42, 50, 57] podle funkce binarni. Zapiš indexy m a porovnávané prvky. Pak funkci spusť. ★☆☆

```
def binarni(s, x):
    lo, hi, kroky = 0, len(s) - 1, 0
    while lo <= hi:
        kroky += 1
        m = (lo + hi) // 2
        if s[m] == x:
            return m, kroky
        elif s[m] < x:
            lo = m + 1
        else:
            hi = m - 1
    return -1, kroky
```

- Zavolej funkci binarni z úlohy 2 pro neseřazený seznam [8, 3, 21, 12, 17] a hledané číslo 3. Co vrátí a proč? ★☆☆
- Seřaď ručně řazením výběrem seznam [29, 10, 14, 37, 13] – zapiš seznam po každém průchodu. Kolik porovnání algoritmus provede? ★☆☆

```
s = [29, 10, 14, 37, 13]
for i in range(len(s) - 1):
    j_min = i
    for j in range(i + 1, len(s)):
        if s[j] < s[j_min]:
            j_min = j
    s[i], s[j_min] = s[j_min], s[i]
print(s)
```

5. Bublínkové řazení s vylepšením: zapiš seznam [5, 1, 4, 2, 8] po každém průchodu. Po kolika průchodech algoritmus skončí a kolik porovnání ušetří proti verzi bez vylepšení? ★★☆☆

```
s = [5, 1, 4, 2, 8]
for k in range(len(s) - 1, 0, -1):
    vymena = False
    for i in range(k):
        if s[i] > s[i + 1]:
            s[i], s[i + 1] = s[i + 1], s[i]
            vymena = True
    print(s)
    if not vymena:
        break
```

6. Algoritmus A potřebuje n kroků, B n^2 kroků a C $\log_2 n$ kroků. Kolikrát víc kroků potřebují, když vstup zvětšíme z 1 000 na 2 000? A z 1 000 na 1 000 000? ★☆☆
7. Vaření ve dvou: těstoviny vařit 10 min, omáčka 15 min, prostřít stůl 5 min, scedit těstoviny 1 min (po uvaření), smíchat 2 min (až je hotová omáčka i scezené těstoviny). Kolik minut trvá práce jednomu, kolik dvěma a proč by třetí pomocník už nepomohl? ★★☆☆
8. Ve Scratchi mají dvě postavy skript „po kliknutí na zelenou vlajku“: postava A „říkej Ahoj! 2 sekundy“, postava B „říkej Nazdar! 2 sekundy“. Obě mluví současně. Jak zařídit, aby B odpověděla až po A? ★☆☆

B. Slovní úlohy

9. Pomocí funkcí `linearni(s, x)` a `binarni(s, x)` (úloha 2) spočítej počet porovnání v seznamu `list(range(n))` pro $n = 1\,000$, $10\,000$, $100\,000$, když hledané číslo v seznamu chybí ($x = n$). Výsledky zapiš do tabulky. ★☆☆

```
def linearni(s, x):
    kroky = 0
    for i in range(len(s)):
        kroky += 1
        if s[i] == x:
            return i, kroky
    return -1, kroky
```

10. Otestuj funkci `razi_vyberem` (porovnej s `sorted` na náhodných datech) a změř její čas pro 1 000, 2 000 a 4 000 náhodných čísel. Kolikrát se čas zvětší při zdvojnásobení n ? Porovnej s vestavěným `sorted()`. ★★☆☆

```
def razeni_vyberem(s):
    s = s[:] # kopie, vstup nezměníme
    for i in range(len(s)):
        j_min = i
        for j in range(i + 1, len(s)):
            if s[j] < s[j_min]:
                j_min = j
        s[i], s[j_min] = s[j_min], s[i]
    return s
```

11. Bublínkové řazení počítá porovnání. Co program vypíše pro už seřazený seznam 1 000 čísel? Vylepši ho: pokud v průchodu nedošlo k žádné výměně, skonči (`break`). Kolik porovnání pak potřebuje seřazený seznam, seznam s jednou prohozenou dvojicí a náhodný seznam? ★★☆☆

```
def bublina(s):
    s = s[:]
    porovnani = 0
    for k in range(len(s) - 1, 0, -1):
        for i in range(k):
            porovnani += 1
            if s[i] > s[i + 1]:
                s[i], s[i + 1] = s[i + 1], s[i]
    return s, porovnani

print(bublina(list(range(1000)))[1])
```

12. Zobecnění: a) Uprav lineární vyhledávání na funkci `najdi_vse(s, x)`, která vrátí seznam všech pozic, kde se x vyskytuje. b) Přidej do `razi_vyberem` parametr `sestupne=False`, který umožní řadit i sestupně. Napiš testy `assert` – i pro seznam `jmen`. ★★☆☆

13. V neseřazeném seznamu 10^6 prvků budeme hledat k -krát. Varianta A: vždy lineárně (průměrně asi 500 000 porovnání). Varianta B: jednou seřadit (asi $2 \cdot 10^7$ porovnání) a pak hledat binárně (20 porovnání). Od kolika hledání je B výhodnější? ★★☆☆
14. Zmenšení 1 200 fotografií trvá na jednom jádru 0,5 s na fotku; načtení seznamu a uložení výsledků trvá 20 s a nelze je rozdělit. Kolik trvá celá úloha na 1, 4 a 8 jádrech? Jaké je zrychlení a jaké by bylo nejvyšší při libovolném počtu jader? ★★☆☆
15. Kolize nad sdílenými daty: Dva procesy chtějí současně přičíst 100 Kč na účet se zůstatkem 1 000 Kč. Každý provede „načti stav – přičti 100 – zapiš“. Program napodobuje jedno nešťastné prostřídání kroků. Co vypíše, co mělo vyjít a jak kolizi zabránit? ★★☆☆

```
stav = 1000
# proces A a proces B chtějí oba přičíst 100 Kč; kroky se prostřídají:
a_precte = stav      # A načte 1000
b_precte = stav      # B načte 1000
stav = a_precte + 100 # A zapiše 1100
stav = b_precte + 100 # B zapiše 1100
print(stav)
```

16. Na dvou procesorech je třeba spustit šest nezávislých úloh s délkami 7, 5, 4, 3, 3, 2 min. Rozděl je tak, aby vše skončilo co nejdříve. Zkus i pravidlo „nejdelší úlohu dej procesoru, který je dříve volný“. ★☆☆

C. Problémy a propojení

17. Propojení s H34: Kolik otázek typu ano/ne potřebuješ, abys uhodl myšlené číslo od 1 do 1 000 000? Jak souvisí binární vyhledávání s množstvím informace v bitech? Kolik různých věcí rozliší hra „20 otázek“? ★★☆☆
18. Vyber a zdůvodni vhodný postup řazení pro: a) 10 čísel ve školní úloze; b) téměř seřazený seznam 10^6 záznamů, kam přibýlo pár nových; c) 10^6 náhodných čísel; d) zařízení s velmi malou pamětí, které nemůže vytvořit kopii dat. ★★☆☆
19. Kombinatorická exploze (H50): Obchodní cestující má objet n měst a vrátit se; různých okruhů je $\frac{(n-1)!}{2}$. Spočítej jejich počet pro $n = 5, 10, 20$. Jak dlouho by trvalo prověřit všechny okruhy pro 20 měst rychlostí 10^9 okruhů za sekundu? A na 1 000 procesorech současně? ★★☆☆
20. Reálná situace: Dva lidé si ve stejné sekundě online rezervují totéž sedadlo v kině. Co se může stát bez řízení souběhu? Jak to rezervační systémy řeší? Uveď i příklad uvážnutí (deadlocku) z běžného života. ★★☆☆

Hotové úlohy k procvičení na internetu

- **Algoritmy.net – Binární vyhledávání** – princip půlení intervalu, složitost $\log_2 n$ proti lineárnímu vyhledávání
<https://www.algoritmy.net/article/21/Binarni-vyhledavani>
- **Algoritmy.net – Selection sort (řazení výběrem)** – princip, ukázka krok za krokem, složitost n^2
<https://www.algoritmy.net/article/4/Selection-sort>
- **Algoritmy.net – Porovnání řadicích algoritmů** – tabulka složitosti a vlastností deseti řadicích algoritmů
<https://www.algoritmy.net/article/75/Porovnani-algoritmu>
- **Bobřík informatiky – archiv testů** – úlohy z algoritmizace a informatického myšlení (kategorie Junior, Senior) k procvičení bez registrace
<https://www.ibobr.cz/test/archiv>

★ základní · ★★ střední · ★★★ náročnější. Vyber si, které úlohy spočítáš; řešení ukládej do svého portfolia.

Výsledky ke kontrole

1. lineární 100; 10 000; 1 000 000; binární 7; 14; 20 ($2^7 = 128$, $2^{14} = 16\,384$, $2^{20} = 1\,048\,576$)
2. 34: $m = 5$ (25), 8 (42), 6 (30), 7 (34) $\rightarrow$ (7, 4); 5: $m = 5$ (25), 2 (12), 0 (3), 1 (8) $\rightarrow$ (-1, 4) – nenalezeno po 4 porovnáních
3. (-1, 2) – „nenalezeno“, i když 3 v seznamu je: po porovnání s 21 zahodí pravou část a po porovnání s 8 i zbytek; binární vyhledávání předpokládá seřazená data
4. [10, 29, 14, 37, 13] $\rightarrow$ [10, 13, 14, 37, 29] $\rightarrow$ [10, 13, 14, 37, 29] $\rightarrow$ [10, 13, 14, 29, 37]; porovnání $4 + 3 + 2 + 1 = 10 = \frac{5 \cdot 4}{2}$
5. [1, 4, 2, 5, 8] $\rightarrow$ [1, 2, 4, 5, 8] $\rightarrow$ [1, 2, 4, 5, 8] (bez výměny, konec); 3 průchody, $4 + 3 + 2 = 9$ porovnání místo 10
6. na 2 000: A $2\times$, B $4\times$, C jen o 1 krok víc ($\approx 10 \rightarrow 11$); na 10^6 : A $1\,000\times$, B $10^6\times$, C $2\times$ ($\approx 10 \rightarrow 20$)
7. jeden 33 min; dva 17 min (A: omáčka 15 + smíchání 2; B: těstoviny 10, scezení 1, stůl 5); zrychlení $\frac{33}{17} \doteq 1,9$; kritická cesta omáčka $\rightarrow$ smíchání má 17 min, třetí člověk ji nezkrátí (H59)
8. A po své větě „pošli zprávu odpověz“ (nebo „pošli zprávu a čekej“), B spustí řeč blokem „po obdržení zprávy odpověz“ – synchronizace souběžných procesů zprávou
9. lineární: 1 000; 10 000; 100 000 (roste s n); binární: 10; 14; 17 (roste o 3–4 při $10\times$ větším n)
10. orientačně 0,02 s; 0,09 s; 0,35 s – při $2\times$ větším n asi $4\times$ déle (kvadraticky); sorted() je stokrát až tisíckrát rychlejší a při $2\times$ větším n trvá asi $2\times$ déle ($n \log_2 n$)
11. 499500; po vylepšení: seřazený 999, jedna prohozená dvojice 1 997 (2 průchody), náhodný stále přes 400 000 – vylepšení pomůže jen u téměř seřazených dat
12. najdi_vse([3, 1, 3, 2, 3], 3) $\rightarrow$ [0, 2, 4], pro chybějící prvek []; v řazení podmínka $s[j] > s[j_min]$, když sestupne, jinak $s[j] < s[j_min]$; funkce funguje i pro text: ["Eva", "Adam", "Cyril"] $\rightarrow$ ["Adam", "Cyril", "Eva"]
13. $500\,000k > 2 \cdot 10^7 + 20k \Rightarrow k > 40 \rightarrow$ zhruba od 40 hledání; pro jedno hledání se řazení nevyplatí
14. $t_1 = 620$ s; $t_4 = 20 + 150 = 170$ s ($S \doteq 3,6$); $t_8 = 20 + 75 = 95$ s ($S \doteq 6,5$); nejvyšší $S = \frac{620}{20} = 31$ – omezuje nedělitelná část
15. 1100 místo 1200 – B pracoval se starou hodnotou; řešení: zámek (proces si účet zamkne od načtení do zápisu, druhý čeká), případně transakce v databázi
16. celkem 24 min $\rightarrow$ ideál 12 min: {7, 5} a {4, 3, 3, 2}; pravidlo nejdelší napřed dá také 12 min (P1: 7, 3, 2; P2: 5, 4, 3); zrychlení 2
17. 20 otázek; každá otázka pólí možnosti = 1 bit, určení 1 z n možností vyžaduje $\log_2 n$ bitů ($\log_2 10^6 \doteq 19,9$); 20 otázek rozliší $2^{20} = 1\,048\,576$ možností
18. a) cokoli jednoduchého a čitelného (výběrem); b) algoritmus, který využije předseřazení – bublinkové s předčasným koncem nebo vestavěné (Timsort je na téměř seřazených datech skoro lineární); c) vestavěné sorted()/sort() ($n \log_2 n$), nikdy n^2 ($\approx 5 \cdot 10^{11}$ porovnání); d) řazení na místě bez kopie (s.sort(), výběrem na místě)
19. 12; 181 440; $\approx 6,1 \cdot 10^{16}$; $\approx 6,1 \cdot 10^7$ s $\doteq 1,9$ roku; na 1 000 procesorech asi 17 hodin – souběh pomůže jen konstantně, explozi nezastaví; prakticky se používají přibližné metody (nejbližší soused)
20. obě rezervace projdou a sedadlo je prodáno dvakrát (kolize: oba přečetli „volno“ před zápisem); řešení: dočasné uzamčení sedadla prvním (na několik minut), transakce v databázi, zpracování požadavků po jednom; deadlock: dvě auta proti sobě v úzké uličce, každé čeká, až druhé couvne

Řešení krok za krokem

Úloha 18

1. a) 10 čísel: na rychlosti nezáleží (desítky kroků) – zvol jednoduchý a čitelný postup, např. řazení výběrem.
2. b) Téměř seřazený seznam: algoritmus, který předseřazení využije – bublinkové řazení s předčasným koncem (málo průchodů) nebo vestavěné sort() (Timsort je na téměř seřazených datech skoro lineární).
3. c) 10^6 náhodných čísel: kvadratický algoritmus by potřeboval asi $\frac{n^2}{2} = 5 \cdot 10^{11}$ porovnání; vestavěné sorted() jen asi $n \log_2 n \approx 2 \cdot 10^7$.
4. d) Malá paměť: řazení na místě bez kopie – s.sort() nebo řazení výběrem, ne sorted(), které vytváří nový seznam.

Úloha 19

1. $n = 5$: $\frac{4!}{2} = 12$; $n = 10$: $\frac{9!}{2} = 181\,440$.

2. $n = 20$: $\frac{19!}{2} \approx 6,1 \cdot 10^{16}$ okruhů.

3. Při 10^9 okruzích za sekundu: $\frac{6,1 \cdot 10^{16}}{10^9} = 6,1 \cdot 10^7$ s.

4. $\frac{6,1 \cdot 10^7}{3\,600 \cdot 24 \cdot 365} \approx 1,9$ roku.

5. Na 1 000 procesorech: $6,1 \cdot 10^4$ s ≈ 17 h.

6. Souběh zrychlí jen konstantně (1 000×), exponenciální růst nezastaví – pro 25 měst by i milion procesorů počítal asi 10 let; prakticky se používají přibližné metody (nejbližší soused).